DNA Version Management: Preserving Directed Selection History Across the Deployment Lifecycle

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 12, 2026

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in "The Instinct/Reasoning Separation Outside the Model" (Li, April 2026), the second paper in the CKS theory series following "Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems" (Li, April 2026).

Abstract

The Coordination Knowledge Substrate (CKS) pattern, as extended by Paper 2, commits to DNA evolution as the mechanism of directed selection: human-governed modifications to the stabilized orchestration content of a cell's DNA layer. Each such modification, authorized under A2.02's modify right and executed through A2.04 rule authoring, changes the behavioral substrate of the cell going forward. Over a deployment's lifecycle, these modifications accumulate — and each prior state of the DNA layer, once changed, represents history that must be preserved. This note formalizes DNA version management as the operational mechanism for tracking and managing that history. The management has seven components: version creation at each modification, version history preservation per A6.02 retroactivity, long-lifecycle version accumulation per A6.14, version inspection per A2.01, behavioral change documentation, compliance demonstration through version history, and cross-level version management across cell, aspect, and Self structural levels per B1.20. The note articulates what makes this management architecturally distinctive versus conventional AI model-level versioning, identifies the biological analog in genetic lineage tracking, enumerates the inherited Paper 1 commitments that ground the mechanism, describes operational implications including rollback as a directed selection event, and states the limits of what DNA version management does and does not cover. B2.69 is the sixty-ninth Phase B2 note and the third of six notes decomposing B1.14 directed selection.

---

1. Why DNA version management requires standalone formalization

Paper 2's directed selection mechanism, established as B1.14 and decomposed beginning at B2.67, commits to DNA evolution as governed modification of the stabilized orchestration substrate. B2.67 established the scope of directed selection — what changes count as DNA modification, what the authority architecture governs, and what the boundaries of the directed selection mechanism are. B2.68 established DNA modification governance — the per-step authority architecture decomposing who

proposes, who authorizes, what verification applies, and what reversion paths exist for each modification event.

What neither note addresses is what happens across the full lifecycle of a deployment that undergoes many such modifications. A single directed selection event produces one DNA modification; a deployment operating over years produces many such events, each changing the behavioral substrate, each building on the prior state. The question that then becomes architecturally load-bearing is: what becomes of the prior DNA states? The answer determines whether the deployment can answer questions such as "what governed behavior at this specific point?" — a question that compliance frameworks, audit processes, and accountability architectures all eventually ask.

DNA version management is the operational mechanism that preserves DNA evolution history. Naming it as a standalone operational variant is warranted because the mechanism is architecturally distinct from the modification governance B2.68 establishes, from the retroactivity treatment B2.70 will formalize, and from the evolution patterns B2.71 will enumerate. Version management is the bookkeeping layer that makes evolution history inspectable, auditable, and compliance-legible across the deployment lifecycle. Without it, a deployment with a rich directed selection history has no substrate-resident record of what that history contains.

The defensive-publication purpose is served by precision: prior art that precisely formalizes each component of a mechanism occupies a larger territory than prior art that gestures at the whole.

2. The mechanism precisely stated

DNA version management in CKS deployments has seven operationally distinct components.

Version creation. Each DNA modification authorized under A2.02's modify right and executed through A2.04 rule authoring creates a new DNA version. A version captures the DNA-layer state at a specific point in the deployment lifecycle. The version record includes: a version identifier (unique across the deployment's history), a creation timestamp, the authoring governance record per A2.40's six provenance metadata fields (who authorized, under what rule, at what time, what changed, what prior state was displaced, and what governance record authorizes the change), and a change record specifying what differed from the prior version. Versions are substrate-resident per A1.08 — version history is itself authoritative state held in the substrate, not in agent memory or external logs.

Version history preservation per A6.02. When a DNA modification creates a new version, the prior version is preserved in the version history rather than overwritten. The new DNA applies forward per A6.02's retroactivity treatment — the new version governs behavior from its creation timestamp onward; it does not alter the historical record of what prior versions governed. Historical versions remain accessible. This preservation is what makes "what governed behavior at time T?" tractably answerable: the answer is the version whose creation timestamp is the most recent one at or before time T.

Long-lifecycle version accumulation per A6.14. Long-lifecycle deployments accumulate many DNA versions. Paper 1's boundary case A6.14 — deployment-evolution rule version compatibility —

addresses the operational challenge this creates: when many DNA versions exist across a long-lived deployment, the compatibility between operational data (Action-layer records created under prior DNA versions) and the currently active DNA version must be managed. DNA version management includes compatibility tracking as a component: the version record is available to diagnose when an Action record was created under a different DNA version than the one currently active. Version management does not resolve compatibility questions automatically; it provides the substrate-resident record that makes compatibility reasoning tractable.

Version inspection per A2.01. Historical DNA versions are inspectable under A2.01's inspect right. Auditors — whether humans exercising governance authority directly or governance processes operating under human direction — can inspect the DNA that governed behavior at any historical point. The inspect right applies not only to the current DNA version but to all preserved historical versions. This is the runtime expression of the "governance is available at all times" commitment A1.01 establishes.

Behavioral change documentation. Each version record documents what behavioral changes the modification introduced — which orchestration rules changed, what prior behavior they specified, and what new behavior they specify. This documentation is not merely descriptive; it serves as the substrate-resident record that compliance and audit processes consume when tracing behavioral evolution. Without behavioral change documentation, version history answers "what changed?" but not "what did that change mean for cell behavior?"

Compliance demonstration. DNA version management taken together — version creation, history preservation, inspection availability, and behavioral change documentation — enables compliance demonstration. The question "what governed this behavior at this time?" is answerable through version history with provenance. Regulated deployments can demonstrate to auditors not only what behavior occurred (from Action-layer records per A2.40 retraceability) but what DNA version authorized and governed that behavior, who authorized that DNA version, and what governance process produced the authorization. Version management is the substrate-side mechanism that makes this answer available without requiring reconstruction from inference.

Cross-level version management. DNA versions exist at cell, aspect, and Self levels per B1.20's recursive inheritance. Cell-level DNA versions capture the orchestration rules governing individual cell behavior. Aspect-level DNA versions capture the orchestration rules governing coordination across aspect-member cells. Self-level DNA versions capture the orchestration rules governing the enterprise brain as a whole. These three sets are managed separately — a change at one level does not automatically create new versions at other levels. However, when a Self-level DNA change affects cell behavior through the expression mechanism per B2.30, the cross-level interaction is documented through cross-reference in the version records of both the Self-level change and the affected cell-level expressions. Cross-reference is the mechanism that makes cross-level version interactions traceable rather than implicit.

Rollback as directed selection. Governance may determine that a directed selection change was mistaken or harmful and may wish to revert to a prior DNA version. Rollback in CKS is not a literal

undo operation that erases the intervening version history. Rollback is a directed selection event: governance uses A2.02's modify right to set the DNA back to a prior version's content, creating a new version event in the version history. The new version records the content of the reverted-to version; the version history preserves the entire sequence including the reverted modifications. This treatment has a compliance consequence: the version history always shows what happened, including the revert, rather than presenting a sanitized view that omits the problematic modifications. Rollback is forward-looking (it governs future behavior from the moment of reversion) while the version history remains a complete record of the backward path.

3. What makes DNA version management architecturally distinctive

The contrast with conventional AI model versioning is load-bearing for understanding what CKS DNA version management contributes that existing practice does not.

Conventional AI system versioning tracks model versions at the model level — model V1, V2, V3. When a model is updated, the prior model version may be archived, but the behavioral specification governing the prior model's outputs (which orchestration rules were active, how they governed specific cell behaviors, who authorized what changes under what governance process) is not tracked at the rule or element level. The version history answers "which model version was deployed?" but not "which orchestration rule, within that deployment, governed this specific behavior at this specific time?"

CKS DNA version management operates at the rule and element level within the DNA layer. The granularity enables a different kind of behavioral audit: not "which model version?" but "which rule version, in which version of the DNA layer, governed this behavior?" The audit can then trace to the governance record that authorized that rule version, to the provenance metadata identifying who authorized it and under what authority, and to the prior version from which the change was made. The behavioral audit is traceable at the rule level, not only at the deployment level.

This granularity is what makes the compliance demonstration described in §2 operationally meaningful. A model-level version record cannot answer "what rule authorized this specific behavior?"; a rule-element-level version record can. The architectural distinctiveness is not in the concept of version management — software version control is a mature practice — but in the application of version management to governed behavioral rules as substrate-resident authoritative state, combined with provenance metadata and the inspection commitment, at every structural level of a CKS deployment.

4. The biological analog

DNA version management in CKS has a biological analog in genetic lineage tracking — genealogy in the broad sense that records how genetic material has changed across generations, tracing which modifications appeared at which generational transitions.

The analog is useful as conceptual scaffold. Just as genealogical records preserve which genetic variants were present at each point in a lineage's history, CKS DNA version management preserves which orchestration rules were present at each point in a deployment's DNA evolution history. Just as genetic lineage tracking enables researchers to determine what genetic material governed an organism's

traits at a given generational point, DNA version management enables auditors to determine what orchestration rules governed a cell's behavior at a given deployment-lifecycle point. The rollback case has a partial analog in de-extinction discussions — the attempt to restore a prior genetic state — but the architectural version (creating a new version whose content matches a prior version) is more precise and operationally cleaner than the biological analog allows.

The architectural substance, however, is not the biological analog but the substrate-resident DNA version history with provenance. The analog functions as a bridge for communication and conceptual orientation; what CKS commits to is the operationalized mechanism: version creation at each A2.04 modification, substrate-resident preservation per A1.08, provenance fields per A2.40, retroactivity treatment per A6.02, inspection per A2.01. Each component carries Paper 1 and Paper 2 commitments rather than biological claims.

5. Inherited Paper 1 commitments

DNA version management inherits directly from seven Paper 1 commitments.

A6.02 (rule retroactivity) — directly load-bearing. Prior DNA versions are preserved in version history; new DNA applies forward, not backward. Version management operationalizes A6.02's retroactivity treatment across the full evolution history. The connection is direct: retroactivity treatment requires that historical versions exist and be distinguished from the current version, which is precisely what version management provides.

A6.14 (deployment-evolution rule version compatibility) — directly load-bearing. Long-lifecycle deployments accumulate many DNA versions, and compatibility between historical Action records and current DNA is a boundary case A6.14 names. Version management includes compatibility tracking as the mechanism that makes A6.14's compatibility reasoning tractable. Without substrate-resident version records, compatibility cannot be assessed systematically.

A2.40 (six provenance metadata fields) — directly relevant. Each version record includes A2.40's six fields applied to the modification event that created the version. Version history provenance is the mechanism that enables compliance demonstration: the governance trail is substrate-resident in the version record, not inferred from external documentation.

A1.07 (path retraceability) — directly relevant. Version management supports retraceability across DNA evolution history. The version history is the substrate-resident record that makes the path from current behavior back to the initial DNA state traceable through all intermediate modifications.

A2.01 (inspect right) — directly relevant. Historical DNA versions are inspectable per A2.01. The inspect right applies to the full version history, not only to the current version. The temporal scope of the inspect right — available at all times — extends to historical versions as a matter of architectural commitment.

A1.08 (substrate as source of truth) — directly relevant. Version records are substrate-resident. The version history is authoritative state held in the substrate, not reconstructed from logs, inferred from operational data, or dependent on agent memory. This places version history within the five categories

of authoritative state Paper 1 §6.2 establishes.

A1.01 (human-governed) — grounding commitment. Version management operates under human governance at all times. The same three rights — inspect, modify, override — that apply to current DNA content apply to version history and to the version management mechanism itself. Governance does not become unavailable over historical version records; auditors and governance authorities retain full access.

6. Operational implications

Several operational implications follow from the formalized mechanism.

Retention policies. Deployments configure version management per retention policies governing how long historical versions are retained before archival or expiration. Long-lifecycle deployments manage potentially large version histories. Retention period is a governance decision; compliance frameworks may specify minimum retention periods for regulated deployments, and these operate as orchestration rules governing the version management mechanism itself.

Version comparison. Version comparison across the history enables systematic understanding of behavioral change between any two DNA states. A deployment can compare version N to version N−5 to understand cumulative behavioral drift across five directed selection events, distinguishing intended from unintended behavioral change.

Cross-level correlation. Cross-level version correlation — tracing how Self-level DNA changes produced cell-level behavioral changes through the expression mechanism — enables understanding of deployment-wide behavioral evolution. Correlation requires cross-references in version records at each affected level, per §2.

Rollback as safety net. Rollback's treatment as a directed selection event (creating a new version rather than erasing history) provides a safety net for problematic modifications without compromising audit integrity. Governance can revert to a prior state without concealing the sequence that led to the revert. The version history remains complete and auditable even after rollback.

Testability per A5.08. DNA version management is testable through A5.08's provenance-completeness test applied to version records: for each DNA modification in the deployment's history, a version record should exist, provenance metadata should be complete, the prior version should be accessible, and behavioral change documentation should be present. Incompleteness in version records is a provenance-completeness violation detectable through this test.

7. Limits

Several limits bound what DNA version management does and does not cover.

Does not prevent future changes. Version management records past modifications; it does not constrain or prevent future directed selection events. Governance authority to modify DNA remains available per A2.02 regardless of how many versions have accumulated. Version management is not a

change-control gate; it is a change-history mechanism.

Does not guarantee behavioral correctness. Version management records what DNA governed behavior at each historical point. It does not guarantee that any version's content was correct, well-designed, or produced intended outcomes. Version history is an evidentiary record, not a quality assurance mechanism.

Rollback is not a literal undo. Rolling back to a prior version creates a new version event, not an erasure of the intervening sequence. The full sequence — including reverted modifications — remains in the version history. Rollback is a governance decision to operate from a prior content state going forward; it does not restore the deployment to a prior state in all respects, and it does not retroactively change what the intervening versions governed.

Versioning is not retroactivity. DNA version management is the mechanism; A6.02 retroactivity treatment is the rule about how new versions apply to prior behavior. B2.70 will formalize retroactivity treatment as a separate note. The two are related but distinct: version management without a retroactivity rule leaves ambiguous how new versions interact with prior Action records; retroactivity treatment without version management governs the rule but has no substrate-resident history to apply it against.

Does not extend to the Action layer. The Action layer accumulates through record creation rather than through version replacement — Action records are appended rather than superseded. DNA version management governs the DNA layer specifically. Action-layer management operates under the accumulation pattern established at B2.26 and is not versioned in the same sense.

Works with A2.40 and A1.07, not independently. Version management is not a complete accountability system in isolation. It works together with A2.40 provenance (which provides per-modification metadata) and A1.07 retraceability (which provides the path-reconstruction commitment). Version management is the substrate-resident history structure; provenance and retraceability are what make that history meaningful for accountability and compliance demonstration.

8. Operational test

A CKS deployment instantiates the DNA version management commitment if and only if all of the following hold: for every DNA modification in the deployment's history, a substrate-resident version record exists that includes a version identifier, a creation timestamp, A2.40 provenance metadata for the modification event, a record of what changed from the prior version, and a reference to the prior version; historical versions are inspectable per A2.01; the current active version is identifiable as the most recent version; cross-level version interactions are cross-referenced in the relevant version records at each affected structural level; and rollback is implemented as a directed selection event creating a new version rather than as erasure of the intervening history.

9. Position in the B1.14 decomposition and Phase B2 progression

B2.69 is the sixty-ninth Phase B2 note. It occupies the third position in the six-note decomposition of B1.14 directed selection:

B2.67 established the scope of directed selection — what changes count as directed selection, how directed selection is distinguished from the other two evolution mechanisms, and what B1.14 covers as an architectural commitment.

B2.68 established DNA modification governance — the per-step authority architecture (who proposes, who authorizes, what verification applies, what reversion paths exist) for each individual directed selection event.

B2.69 (this note) formalizes DNA version management — the operational mechanism for tracking and managing the accumulation of DNA versions across the full directed selection evolution lifecycle, preserving history, enabling compliance demonstration, and supporting cross-level traceability.

B2.70 (next) will formalize retroactivity treatment for DNA changes — how new DNA versions apply to prior Action-layer records and operational data, the governing rule that works in concert with the version history mechanism this note establishes.

B2.71 will enumerate directed selection evolution patterns — the recurring patterns of directed selection events that deployments exhibit across their lifecycles.

B2.72 will formalize directed selection verification — the verification mechanisms governance applies to directed selection events before and after acceptance.

After B2.72 closes the B1.14 decomposition, Phase B2 continues with the B1.15 action-feedback decomposition beginning at B2.73.

Naming DNA version management as a standalone note establishes public prior art for the combination of substrate-resident version history, rule-element-level granularity, retroactivity-grounded preservation, provenance-complete modification records, rollback as directed selection creating a new version, cross-level version correlation, and testability through provenance-completeness testing. Each component is derivable from Paper 2’s directed selection mechanism and Paper 1’s inherited commitments; formalizing them together as a named operational variant occupies the territory where any party would otherwise assert that this specific combination is novel.

Source papers

Li, Wenxin. “The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance.” April 2026.

Li, Wenxin. “Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.” April 2026.